

Received 12 September 2024, accepted 27 September 2024, date of publication 2 October 2024,
date of current version 16 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3472473

RESEARCH ARTICLE

Comparative Analysis of A3C and PPO Algorithms in Reinforcement Learning: A Survey on General Environments

ALBERTO DEL RIO¹, DAVID JIMENEZ², AND JAVIER SERRANO³

¹Signals, Systems and Radiocommunications Department, Escuela Técnica Superior de Ingenieros de Telecomunicación (ETSIT), Universidad Politécnica de Madrid, 28040 Madrid, Spain

²Physical Electronics, Electrical Engineering and Applied Physics Department, Escuela Técnica Superior de Ingenieros de Telecomunicación (ETSIT), Universidad Politécnica de Madrid, 28040 Madrid, Spain

³Informatic Systems Department, Escuela Técnica Superior de Ingeniería de Sistemas Informáticos (ETSISI), Universidad Politécnica de Madrid, 28031 Madrid, Spain

Corresponding author: Alberto del Rio (a.delriop@upm.es)

This work was supported in part by the Horizon Europe CODECO Project under Grant 101092696, in part by the Horizon Europe NEMO Project under Grant 101070118, and in part by the UNICO-5G I+D (B5GEMINI-AIUC) Project funded by the Ministry of Economic Affairs and Digital Transformation of the Spanish Government and the NextGenerationEU (Recovery, Transformation and Resilience Plan–PRTR) under Grant TSI063000-2021-79.

ABSTRACT This research article presents a comparison between two mainstream Deep Reinforcement Learning (DRL) algorithms, Asynchronous Advantage Actor-Critic (A3C) and Proximal Policy Optimization (PPO), in the context of two diverse environments: CartPole and Lunar Lander. DRL algorithms are widely known for their effectiveness in training agents to navigate complex environments and achieve optimal policies. Nevertheless, a methodical assessment of their effectiveness in various settings is crucial for comprehending their advantages and disadvantages. In this study, we conduct experiments on the CartPole and Lunar Lander environments using both A3C and PPO algorithms. We compare their performance in terms of convergence speed and stability. Our results indicate that A3C typically achieves quicker training times, but exhibits greater instability in reward values. Conversely, PPO demonstrates a more stable training process at the expense of longer execution times. An evaluation of the environment is needed in terms of algorithm selection, based on specific application needs, balancing between training time and stability. A3C is ideal for applications requiring rapid training, while PPO is better suited for those prioritizing training stability.

INDEX TERMS A3C, CartPole, comparison, environment complexity, Lunar Lander, performance analysis, PPO, reinforcement learning, sample efficiency, stability.

I. INTRODUCTION

Reinforcement Learning (RL) has emerged as a powerful paradigm for training intelligent agents to make sequential decisions in complex and dynamic environments where adaptive decision-making is required. RL's ability to learn from interactions and optimize actions based on feedback makes it a powerful tool for solving this kind of environments. Unlike supervised learning, where agents learn from labeled

training datasets, RL agents learn through trial and error, receiving feedback in the form of rewards or penalties based on their actions, with the ultimate goal of maximizing cumulative rewards over time. Fig. 1 represents the interactive loop between agent and the environment.

This feedback loop enables agents to discover optimal strategies for achieving long-term goals in various domains, ranging from gaming [1], [2], [3], [4], [5] to finance [6], [7], robotic control [8], [9], [10], media optimization [11], [12], [13], [14], energy management [15], [16], computer vision [17], [18], recommendation systems [19], [20], [21],

The associate editor coordinating the review of this manuscript and approving it for publication was Abdullah Iliyasu¹.

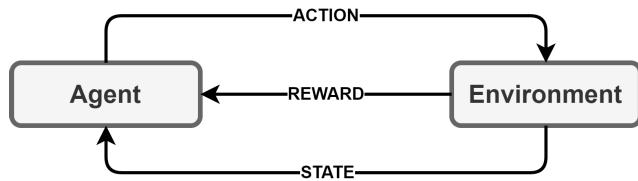


FIGURE 1. Reinforcement learning framework.

autonomous driving [22], [23], [24], [25], [26] and health-care [27], [28]. Additionally, recent advances in RL, and by using latest algorithms, it has significantly contributed to fine tune Large Language Models (LLM) [29], [30] using human feedback.

Two prominent RL algorithms that have garnered significant attention in recent years are Asynchronous Advantage Actor-Critic (A3C) [31] and Proximal Policy Optimization (PPO) [32]. A3C is an actor-critic algorithm that was introduced by researchers at DeepMind in 2016 and enhances the stability and performance of traditional RL methods by using asynchronous gradient descent for optimization of deep neural network controllers; it employs a distributed architecture to parallelize the training process, enabling faster convergence. On the other hand, PPO, introduced by OpenAI in 2017, is a policy gradient method that simplifies and improves the reliability of training RL agents. It seeks to balance the need for exploration and exploitation while ensuring stable updates. It utilizes a clipping mechanism to constrain policy updates within a trust region, thus preventing large, destabilizing changes to the policy. This approach helps maintain stable and consistent performance during training.

While in general, DRL algorithms have demonstrated impressive results across a wide range of tasks, specific algorithms have their strengths and are suitable for different types of environment problems. The choice between them depends on the specific requirements of the task, such as the need for fast convergence, stability, ease of implementation, and computational resources. Evaluating their performance across diverse environments is crucial for comprehending their respective advantages and disadvantages. Additionally, the influence of environment complexity on algorithm performance remains an open question that warrants investigation [33].

In this study, we aim to provide a comparison between the A3C and PPO algorithms in the context of two distinct environments based on OpenAI Gym [34]: Cart Pole [35] and Lunar Lander. The Cart Pole environment, a classic benchmark problem in RL, involves balancing a pole on a cart by applying left or right forces. In contrast, the Lunar Lander environment presents a more complex scenario where an agent must navigate a lunar lander to safely touch down on the moon's surface.

We analyze the performance of these algorithms with respect to convergence speed, stability, and sample efficiency, with a focus on the trade-offs between training speed

and stability. Additionally, we assess the scalability of the A3C algorithm, in comparison with PPO with one agent, by evaluating the impact of different worker counts on iteration speed, offering understanding on how to enhance computational efficiency in environments where parallel training is performed.

II. RELATED WORK

Many researchers have explored the effectiveness of Deep Reinforcement Learning (DRL) algorithms, such as Deep Q-Networks (DQN) [36], [37], [38], [39], Trust Region Policy Optimization (TRPO) [40], [41], [42], and the aforementioned A3C and PPO algorithms. These studies often evaluate algorithm performance on standard benchmark environments, such as Atari games [43] and Mujoco simulations [44].

Beyond A3C and PPO, other methods like model predictive control (MPC) are compared to RL, solving supervised learning problems, showing good results in several works [45], [46]. Other studies have explored different RL approaches, such as comparing DQN and A3C in the context of Atari games, revealing A3C's superior training time and stability [47]. Further comparative studies on RL algorithms applied to the Cart-Pole problem applying Q-learning and actor-critic methods [48]. In addition, a work integrated fuzzy logic system with SARSA and Q-learning algorithms to scale up/down cloud resources to meet QoS requirements while reducing costs [49]. Furthermore, a survey on dynamical environments emphasized the importance of continual learning to adapt to different conditions [50].

Recent studies have conducted comparisons between A3C, PPO and other state-of-the-art RL algorithms across diverse application domains. For instance, some examples include dynamic pricing optimization in comparison to human level [51]; Actor-Critic algorithms for stock trading environments [52]; docking control of autonomous underwater vehicle control comparing PPO, TD3 and PPO [53]; robotic positioning on NVIDIA Omniverse comparing several algorithms [54]; and even NB-IoT resource allocation mechanism, with and optimization not only using A3C and PPO, but expanding the research to multi-agent in the case of PPO algorithm [55].

Some comparative studies research deep into specific aspects of algorithm behavior, such as convergence speed [56], [57], but in general, new models have been developed to improve sample efficiency [58], [59], [60] and stability [61], [62], [63]. Aligned with efficiency, the development of the neural networks included in RL significantly affects the convergence and stability of algorithms [64]. RL for neural architecture search (NAS) has automated neural network design, boosting performance and efficiency with techniques like performance prediction and simplified proxy metrics [65].

Exploration strategies are crucial in RL to balance exploration and exploitation [66]. The epsilon-greedy (ϵ -greedy)

strategy, which randomly selects actions occasionally, is common but can be inefficient due to its randomness [67]. It was studied to combine softmax and ϵ -greedy methods to balance exploration and exploitation effectively by adjusting the probability of selecting each action based on their estimated values [68]. A work applied this method to robot control, demonstrating real applicability [69]. Count-based strategies use state-action visit counts to dynamically adjust exploration, encouraging visits to less-explored states and addressing sparse rewards [70], [71].

Additionally, related work also includes investigations into algorithmic extensions. Researchers have proposed modifications to existing algorithms to address specific challenges. These modifications may involve including additional tools for incorporating memory mechanisms [72], [73], [74], or introducing regularization techniques to prevent overfitting [75].

Researchers are also interested in how well learned behaviors (policies) can be applied to new situations (tasks and environments). Transfer learning helps achieve this by using knowledge from one task to learn a new, related task faster [76]. Highlighting methods such as policy transfer [77], [78], value function transfer [79], and model transfer [80], [81], [82]. This is crucial for scaling up RL algorithms to handle the complexity and variety of real-world problems [83], [84], or including enhances to improve cold-start training [85], [86].

Incorporating advanced control strategies from multi agent systems could significantly enhance RL frameworks. For example, a study on cooperative output regulation introduced adaptive methods that allow for the real-time adjustment of system parameters to achieve accurate output regulation and improved system performance [87]. Integrating such adaptive mechanisms into RL could enable dynamic tuning of learning parameters, thereby enhancing the algorithm's responsiveness to changing environments. Additionally, research on event-triggered control demonstrated the use of prescribed-time strategies to ensure that system objectives are met within a predefined time frame [88], that is, imposing constraints on transient performance to achieve the outcomes within specific time limits.

III. METHODOLOGY

A. EXPERIMENTAL SETUP

The study was carried out using a computational setup comprising an Intel(R) Core(TM) i9-10900 CPU running at 2.80GHz and 32GB of RAM. The experiments were performed on Ubuntu 22.04.2 LTS to ensure a reliable environment. For GPU-accelerated tasks, we utilized an Nvidia 2080 Ti graphics card along with up-to-date drivers to maximize efficiency. Moreover, CUDA version 12.7 was integrated to leverage GPU parallelism for faster training of DRL models. All information is summarized in Table 1.

TABLE 1. Hardware and software configurations.

Hardware Component	Specification
CPU	Intel(R) Core(TM) i9-10900 CPU @ 2.80GHz
RAM	32GB
Operating System	Ubuntu 22.04.2 LTS
GPU	Nvidia 2080 Ti
GPU Drivers	537.42
CUDA Toolkit	CUDA 12.7

B. REINFORCEMENT LEARNING OVERVIEW

RL [89] is a machine learning paradigm where an agent learns to interact with an environment to achieve a specific goal by taking actions and receiving feedback in the form of rewards. The agent's objective is to learn a policy that maps states to actions in a way that maximizes cumulative rewards over time. RL algorithms can broadly be categorized into two types: value-based methods, which aim to estimate the value function associated with each state-action pair, and policy-based methods, which directly parameterize the agent's policy.

On-policy algorithms learn the value of the policy being executed by the agent, meaning they update the policy based on actions taken from the current policy. Off-policy algorithms, by contrast, learn the value of the optimal policy independently of the agent's actions, allowing them to learn from actions outside the current policy.

1) A3C

A3C is an on-policy RL algorithm, an asynchronous, parallelized version of the Advantage Actor-Critic (A2C) algorithm, designed to concurrently learn an approximation of the value function (the Critic) and the policy (the Actor). A3C uses multiple parallel agents to explore the environment and update a shared model, which enhances exploration and learning efficiency. It combines ideas from asynchronous training and the advantage function to achieve efficient and scalable learning. Additionally, the use of multiple agents in parallel, enable faster convergence.

A3C maintains two separate neural networks: an Actor network $\pi(a|s; \theta)$ that outputs the policy, and a critic network $V(s; \theta_v)$ that estimates the value function. The Actor and critic are updated asynchronously using parallel workers. Algorithm 1 shows the details of the training.

The core of A3C is the Actor-Critic architecture, which integrates two components:

- **Actor (Policy Network):** The Actor is responsible for learning and representing the policy, $\pi(a|s)$, which defines the probability distribution over actions given states. Typically, this policy is parameterized by a neural network with trainable weights denoted as θ . The Actor aims to maximize the expected cumulative reward by selecting actions that lead to higher expected returns.
- **Critic (Value Network):** The Critic estimates the value function, $V(s)$, which quantifies the expected cumulative reward starting from a given state s and following the

Algorithm 1 Asynchronous Advantage Actor-Critic (A3C)

```

1: Initialize global shared parameters  $\theta$  and  $\theta_v$ 
2: Initialize global counter  $T \leftarrow 0$ 
3: Initialize thread-specific parameters  $\theta'$  and  $\theta'_v$ 
4: while  $T < T_{max}$  do
5:   Reset gradients:  $d\theta \leftarrow 0$  and  $d\theta_v \leftarrow 0$ 
6:   Synchronize thread-specific parameters:  $\theta' \leftarrow \theta$  and  $\theta'_v \leftarrow \theta_v$ 
7:    $t_{start} \leftarrow t$ 
8:   Get initial state  $s_t$ 
9:   repeat
10:    Perform action  $a_t$  according to policy  $\pi(a_t|s_t; \theta')$ 
11:    Receive reward  $r_t$  and new state  $s_{t+1}$ 
12:     $t \leftarrow t + 1$ 
13:     $T \leftarrow T + 1$ 
14:  until terminal  $s_t$  or  $t - t_{start} == t_{max}$ 
15:   $R \leftarrow \begin{cases} r_{end} & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \end{cases}$ 
16:  for  $i \in \{t - 1, \dots, t_{start}\}$  do
17:     $R \leftarrow r_i + \gamma R$ 
18:    Accumulate gradients wrt  $\theta'$ :
        
$$d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i; \theta')(R - V(s_i; \theta'_v))$$

19:    Accumulate gradients wrt  $\theta'_v$ :
        
$$d\theta_v \leftarrow d\theta_v + \nabla_{\theta'_v} (R - V(s_i; \theta'_v))^2$$

20:  end for
21:  Perform asynchronous update of  $\theta$  using  $d\theta$ 
22:  Perform asynchronous update of  $\theta_v$  using  $d\theta_v$ 
23: end while

```

policy π . The value function serves as a baseline for the Actor, helping it assess the quality of its chosen actions. The Critic's goal is to minimize the temporal difference (TD) error, which measures the discrepancy between predicted and actual rewards.

A key concept in A3C is the advantage function in (1), denoted as $A(s, a)$, which quantifies the advantage of taking a specific action a in state s compared to the average action chosen under the policy π . Mathematically, it is defined as:

$$A(s, a) = Q(s, a) - V(s) \quad (1)$$

where:

- $Q(s, a)$ represents the estimated action-value (Q-value) of taking action a in state s .
- $V(s)$ is the estimated value of state s .

The advantage function provides crucial information for the Actor to adjust its policy for the reduction of variance, while improving learning stability. Positive advantages indicate actions that are better than the expected, while negative advantages imply actions that are worse.

The A3C algorithm employs an asynchronous training paradigm where multiple agents (threads) interact with the

environment simultaneously which improves exploration and learning efficiency. Each agent collects experiences by executing actions according to its policy and stores these experiences.

The training objective for the Actor is to maximize the expected cumulative reward, which is achieved through policy gradient methods. The Critic aims to minimize the TD error between predicted and actual rewards.

The asynchronous nature of A3C facilitates exploration and accelerates learning, making it well-suited for parallelized computation on multi-core CPUs or GPUs, or even parallel environments without the need of additional techniques like transfer or federated learning.

C. PPO ALGORITHM**Algorithm 2** Proximal Policy Optimization (PPO)

```

1: Initialize policy parameters  $\theta$ 
2: Initialize value function parameters  $\phi$ 
3: for each iteration do
4:   Collect a set of trajectories  $\mathcal{D}$  by running policy  $\pi_\theta$  in the environment
5:   for each trajectory do
6:     for each time step  $t$  do
7:       Compute advantage estimate  $\hat{A}_t$ 
8:     end for
9:   end for
10:  Update the policy by maximizing the PPO-Clip objective:
        
$$\mathcal{L}^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[ \min \left( \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t, \text{clip} \left( \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

11:  Update the value function by minimizing the loss:
        
$$\mathcal{L}^{VF}(\phi) = \hat{\mathbb{E}}_t \left[ (V_\phi(s_t) - R_t)^2 \right]$$

12:  Update the policy parameters  $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}^{CLIP}(\theta)$ 
13:  Update the value function parameters  $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}^{VF}(\phi)$ 
14: end for

```

PPO is an on-policy RL algorithm that addresses some of the limitations of previous policy optimization methods, such as high variance and instability during training. PPO belongs to the family of Trust Region Policy Optimization (TRPO) algorithms and aims to ensure stable and efficient policy updates while maximizing cumulative rewards, balancing simplicity, performance and ease of implementation. Algorithm 2 shows the details of the training.

The core idea behind PPO is to enforce a constraint on the policy update to prevent large policy changes that could lead to instability. This is achieved through the use of a

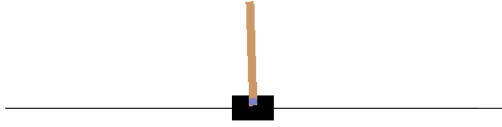


FIGURE 2. CartPole.

clipped objective function, which ensures that the updates to the policy do not deviate too much from the current policy. Instead of directly optimizing the policy parameters to maximize rewards, PPO seeks to find a balance between exploration and exploitation by limiting the extent of policy updates.

The main components of PPO include:

- **Policy Network:** The policy network, denoted as $\pi_\theta(a|s)$, parameterizes the agent's policy, where θ represents the trainable weights of the neural network. The policy network outputs a probability distribution over actions given states, allowing the agent to select actions based on the learned policy.

- **Value Network:** PPO also incorporates a value network, denoted as $V_\phi(s)$, which estimates the value function for states. The value network helps the agent assess the expected cumulative rewards associated with different states and guides policy updates. The value function is typically estimated using a separate neural network with parameters ϕ .

PPO introduces a surrogate objective function (referenced in (2)), that constrains the policy update to be within a specified range. The objective function is given by the clipped surrogate objective:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2)$$

where $\hat{\mathbb{E}}_t$ represents the empirical expectation over a batch of experiences sampled from the environment. $r_t(\theta)$ denotes the ratio of the probability of selecting the action under the updated policy to the probability of selecting the action under the old policy. $\hat{A}_t$ represents the advantage estimate for the action taken at time step t . ϵ is a hyperparameter that controls the extent of policy updates.

By clipping the surrogate objective, PPO ensures that the policy update does not deviate too far from the old policy, thus preventing large policy changes that could destabilize training.

PPO utilizes an iterative optimization process where the policy and value networks are updated using mini-batch gradient descent based on the clipped surrogate objective. Additionally, PPO incorporates entropy regularization, by introducing an entropy term in the objective function to encourage exploration and prevent premature convergence to suboptimal policies.

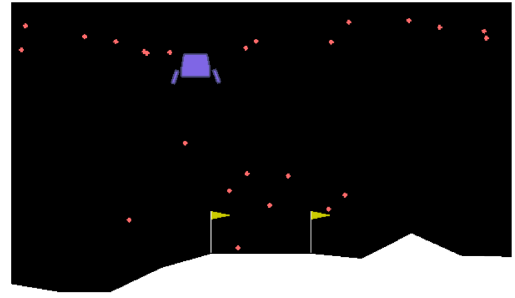


FIGURE 3. Lunar lander.

D. GENERAL ENVIRONMENTS

OpenAI's Gym library serves as the foundational source for our test environments. It encompasses a diverse set of environments, spanning classic control tasks, Atari games, MuJoCo simulations, and more. These environments have been widely adopted in the RL community, offering standardized benchmarks for algorithm evaluation.

This work employs a controlled environment approach, utilizing a set of distinct experimental conditions to conduct the tests and obtain the results. These environments vary in their convergence criteria, including the number of episodes required and the associated time frame for achieving convergence:

- 1) The CartPole environment (see Fig. 2) takes the example of the inverted pendulum problem, where the objective is to balance a pole on a moving cart. The agent must apply forces to the cart to keep the pole upright for as long as possible. It is encompassed on classic control environments, which are simple physics-based tasks that are often used as the basic benchmark problems for RL algorithms.
 - **Action space:** Discrete(2). The agent can either apply a force to move the cart to the left or to the right.
 - **Observation shape:** (4,). The state space is represented by four variables: cart position, cart velocity, pole angle, and pole velocity at the tip.
 - **Reward Structure:** The agent receives a reward of +1 for every time step that the pole remains balanced. The episode terminates if the pole falls beyond a certain angle or the cart moves out of the boundary.
 - **Reward Threshold:** 475. A score above this threshold is considered a success.
- 2) The Lunar Lander environment (see Fig. 3) simulates the task of landing a spacecraft on the moon's surface. The agent must control the spacecraft's orientation and thrust to achieve a safe landing. The environment is part of the Box 2D environments which provide more complex physics simulations for RL tasks.
 - **Action Space:** Discrete(4). The agent can choose to do nothing, fire the left orientation engine,

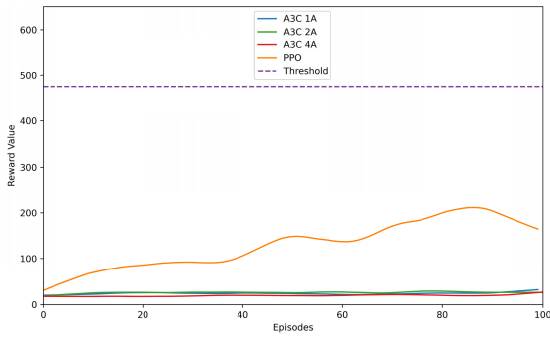


FIGURE 4. Cartpole 100.

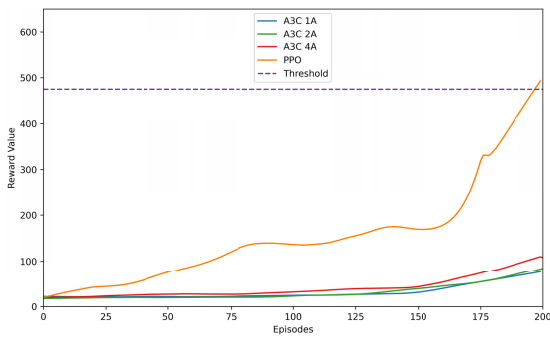


FIGURE 5. Cartpole 200.

fire the right orientation engine, or fire the main engine.

- Observation Shape: (8,). The state space includes the lander's position (x, y), velocity (x, y), angle, angular velocity, and indicators of leg contact with the ground.
- Reward Structure: The agent is rewarded based on landing to the landing pad (+100 points, +10 points per leg contact) and penalties for crashing (−100 points) or excessive fuel usage (−0.3 for firing main engine, −0.003 for firing side engine).
- Reward Threshold: 200. Scores above this threshold indicate a successful landing.

The environments can be seamlessly imported into the workflow using simple lines of code. These environments have undergone multiple iterations and continuous improvements over time. Consequently, they were selected for evaluating the efficacy of the developed algorithms.

While some environments necessitate minimal learning with a smaller number of episodes, thereby requiring only a single agent for training, others present greater complexity, demanding a higher number of iterations to converge towards an optimal model. Hence, alongside analyzing reward values, one crucial aspect to consider is the training time required for each environment.

IV. RESULTS

In line with previous use cases, the algorithms are assessed based on the reward values attained during training. The

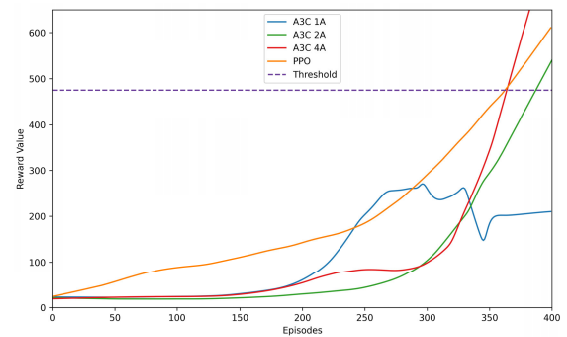


FIGURE 6. Cartpole 400.

extracted results provide a comparative analysis of the algorithms, utilizing a single agent for the PPO algorithm. Additionally, for the A3C algorithm, results are presented for one, two, and four agents. Notably, within the A3C algorithm section, emphasis is placed on the potential benefits of parallel and asynchronous training, which can enhance both training times and performance. At times, algorithms may tend towards excessive conservatism or exploration of ineffective areas, making additional agents valuable for exploring new actions and redirecting training strategies.

We begin our experiments with the CartPole environment, a classic control benchmark for algorithms evaluation. This environment offers a simple setting, allowing for straightforward implementation and evaluation. We trained both A3C and PPO on the CartPole environment and evaluated their performance in terms of accumulated reward over 100, 200, and 400 episodes. The CartPole environment has a reward threshold of 475, which indicates successful balancing of the cart-pole system after this threshold has been surpassed.

For short episodes (100), neither A3C nor PPO achieved optimal performance (Fig. 4). PPO exhibited a more stable learning curve, but still fell short of the reward threshold. A3C showed significant improvement in the 200-episode range (Fig. 5), with PPO achieving the threshold performance.

By the 400-episode mark (Fig. 6), both algorithms surpassed the reward threshold, except A3C using one agent. A3C with multiple agents matched the performance of PPO, demonstrating no significant difference in terms of reward. It is worth noting that the A3C model with four agents experienced exponential reward growth after approximately 350 episodes.

A3C's parallel agent architecture led to significant improvements in training time, as shown in Fig. 7. For 100 episodes, A3C completed training in under 10 seconds in all cases, while PPO (with a single agent) took around 75 seconds to reach the 100-episodes threshold. For 400 episodes, A3C's training time with a single agent increased to over 90 seconds, but it was reduced to around 70 seconds with two and four agents. PPO's training time, on the other hand, increased significantly to an average of 452 seconds.

Fig. 8 shows the percentage of episodes where the accumulated reward exceeded the threshold. PPO outperformed

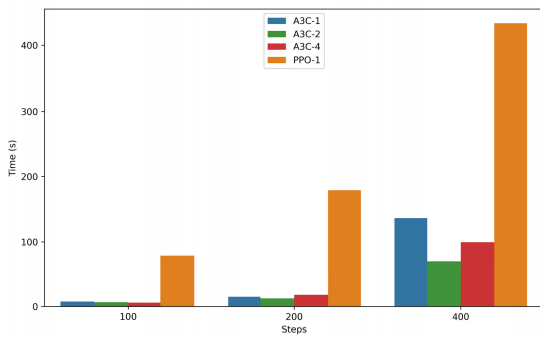


FIGURE 7. Cart pole time.

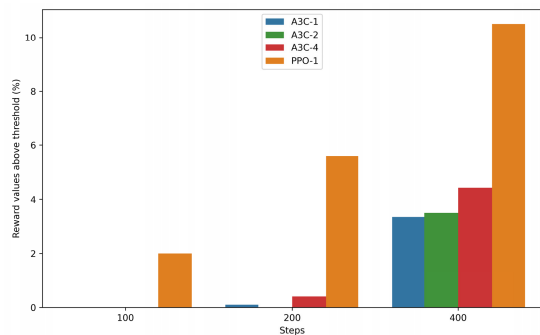


FIGURE 8. Cart pole threshold.

A3C in all cases. The best performance achieved by A3C was 4.45% of episodes above the threshold, while PPO doubled that value to 10.50%.

In addition to Cartpole, we evaluated the performance of A3C and PPO on the LunarLander environment, which is more complex than CartPole. It has a larger state space and a more challenging objective: landing the spacecraft within the designated landing zone. In this environment, landing the spacecraft on the ground is rewarded with 100-140 points, and the threshold for successful completion is around 200 points.

While previous cases in the CartPole environment might have required between 100 and 400 episodes to yield results, in this case, we must approach around 1.000 episodes to observe certain learning. Additionally, if we increase this value to around 2.000 episodes, both algorithms show a learning trend, requiring around 5.000 episodes to surpass the reward barrier and achieve convergence.

Results for Lunar Lander environment are expressed for episodes counts of 1.000, 2.000, and 5.000. For the 1.000-episode case (Fig. 9), we observe a rising trend in learning, with mean reward values nearing positive figures. A similar situation is observed for the 2.000-episode case, where all algorithms exhibit a similar trend, reaching reward values of 100, indicating proximity to learning. Finally, at the 5.000-episode mark, algorithms begin to struggle to achieve high rewards and reach the 200 threshold. Although, as previously discussed, above values of 100, we find the system capable of landing the spacecraft, but it tends to undertake actions that hinder surpassing the threshold on

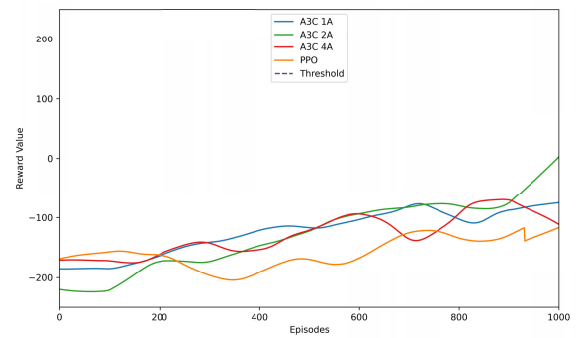


FIGURE 9. Lunar lander 1000.

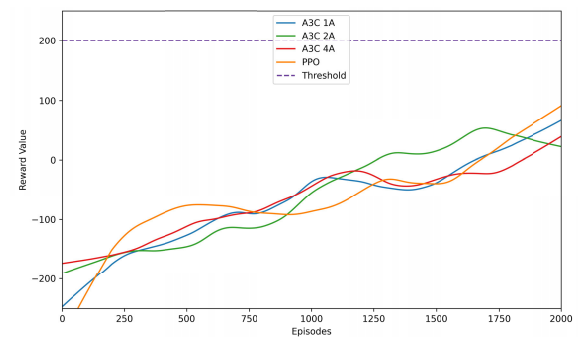


FIGURE 10. Lunar lander 2000.

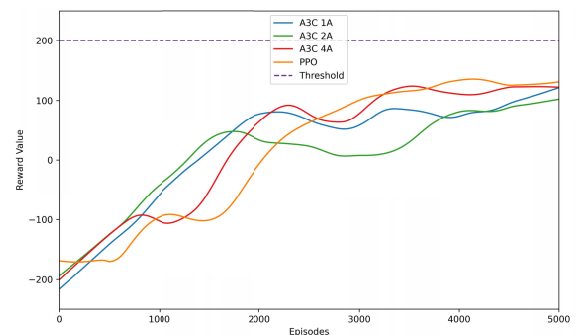


FIGURE 11. Lunar lander 5000.

average. However, specifically in Fig. 13, we observe that many iterations surpass the threshold, indicating a flaw in the smoothness of the previous figure for not showing any algorithm to surpass the threshold.

The time taken by the algorithms can be observed in Fig. 12, where this time, they are approximately similar among them. Particularly noteworthy is A3C with four agents, which manages to reduce the training time in most cases. For the 5.000-episode range, PPO stands out negatively, showing an average of approximately 17.500 seconds needed to complete the training.

As an additional note, in the initial stages of training, the algorithm utilizes between 50-100 steps to complete an episode, but as it progresses and converges, this value increases to around 300-400 to surpass the reward threshold. However, during exploratory moments, the algorithm seeks

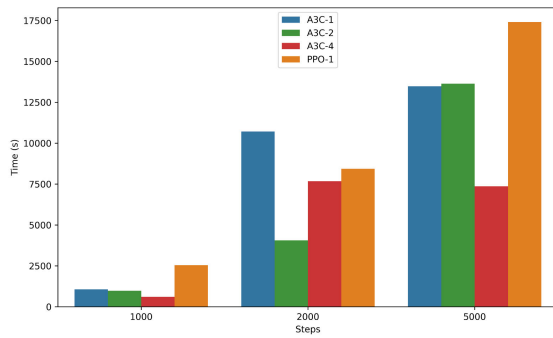


FIGURE 12. Lunar lander time.

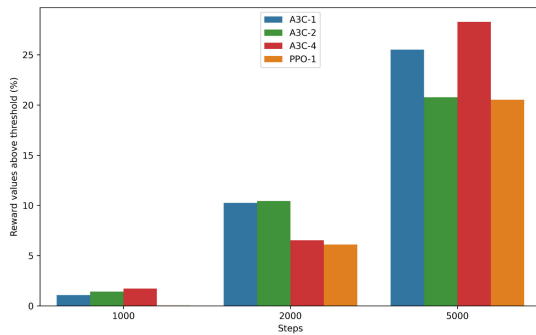


FIGURE 13. Lunar lander threshold.

to be as conservative as possible, without risking, leading to taking more than 50.000 steps in certain situations. These extreme cases result in an extraordinary increase in training time and not only that, but they also detrimentally affect the training, as the rewards obtained in these episodes exceeded -4.000 . These moments have significantly increased training times for the algorithm, as they generally block the agent from training for a long period, especially in cases like A3C with a single agent, and PPO.

Ultimately, the values surpassing the threshold exhibit positivity compared to the CartPole scenario. For instance, in the 2.000-episode range, there is already a 10% occurrence of values exceeding the threshold, exponentially increasing in the case of 5.000 episodes, where all algorithms surpass the 20% mark. All these data is summarized in Fig. 13.

Finally, to complete the comparison between the algorithms, we show a comparison for the two environments. We compare the results of the evolution of the average reward over time. That is, it shows how long each algorithm takes to train, averaging the values, and observing how long it takes to reach the reward thresholds. As an additional note, these graphs only show the data for the trainings with the highest number of episodes, i.e. 400 for Cart Pole and 5.000 for Lunar Lander. Since we obtain an average reward between the five trainings of each model, it was decided to limit the time of the time axis to the average time between the five trainings, for all algorithms.

In Fig. 14, referring to the Cart Pole training, we can see graphically how the difference in time between A3C in all

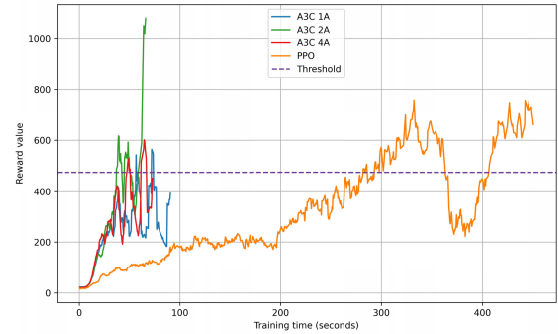


FIGURE 14. Cart pole reward over time.

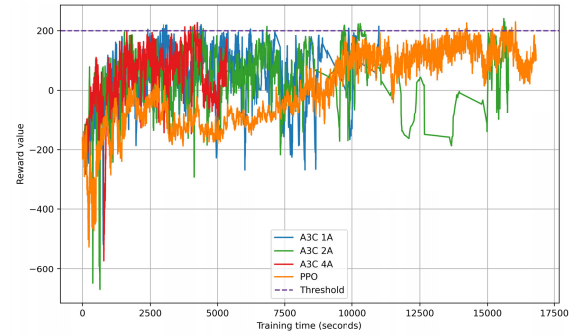


FIGURE 15. Lunar lander reward over time.

its agent configurations and PPO is notable. Generally, A3C manages to exceed the reward threshold in early episodes, but with quite a few value spikes. In contrast, PPO shows a training that requires more time (up to 4 times more), but shows a training that is not only more stable (except for a high decline in advanced training times), but also has higher reward thresholds and is clearly in a state of ascent.

On the other hand, there is the training on Lunar Lander, in Fig. 15. This figure requires more attention since, in general, training times are longer. Again, the A3C algorithm manages to reach high reward levels in short times. However, the reward variations for A3C in 1 and 2 agent configurations are high over time, being smaller in the case of 4 agents. Additionally, an erratic training is seen in A3C with 2 agents in its last stages of training. Finally, PPO, as in the previous environment, exhibits a greater delay in training, but once it reaches a close convergence, we observe that the reward variations between close times are much smaller than with respect to A3C. Not only that, but it is rare to see rewards below 0 in the final stages of training.

In Table 2, we provide a summary of the performance metrics for the A3C and PPO algorithms across all agent configurations, encompassing episodes taken, training time, and the percentage of values surpassing the predefined threshold. It is important to note that all results presented in the table represent average values obtained from five distinct training runs for each configuration. This approach ensures a more robust assessment of the algorithms' performance

TABLE 2. Summary results.

Environment	Alg.	Agents (n)	Epds.	Time (s)	Best rwd.	Rwd. thr. (%)
CartPole	A3C	1	100	6	96	0,00
			200	10	80	0,10
			400	92	7.290	3,35
		2	100	5	65	0,00
			200	12	135	0,00
			400	67	2.434	3,50
	PPO	4	100	5	75	0,00
			200	15	421	0,40
			400	73	1.738	4,45
		1	100	75	467	2,00
			200	124	315	5,60
			400	452	1.375	10,50
LunarLander	A3C	1	1.000	831	244	3,70
			2.000	10.544	286	18,43
			5.000	11.001	314	39,29
		2	1.000	607	289	4,56
			2.000	2.627	284	19,24
			5.000	15.771	308	33,88
	PPO	4	1.000	585	276	4,24
			2.000	7.842	300	14,42
			5.000	5.332	319	42,09
		1	1.000	1.887	176	0,22
			2.000	8.958	290	14,00
			5.000	16.833	297	33,91

and offers a clearer understanding of their capabilities under varying conditions.

V. DISCUSSION

The observed performance differences between A3C and PPO in our experiments can be attributed to several underlying factors related to the design and operational mechanisms of the two algorithms. A3C, as an asynchronous actor-critic algorithm, leverages multiple parallel workers to explore the environment simultaneously. This parallelism can accelerate the learning process, leading to faster convergence in environments with relatively simple dynamics, as seen in the CartPole environment. However, this asynchronous approach can also introduce instability in the training process, as the global model parameters are updated based on gradients computed from different workers, which may be at varying stages of exploration and exploitation. This can result in the oscillations and instability observed in the reward values for A3C.

In contrast, PPO, which utilizes a clipped surrogate objective to constrain policy updates, focuses on maintaining stable and incremental improvements to the policy. This approach mitigates the risk of large, destabilizing updates, leading to a more consistent and stable learning process, especially in more complex environments like Lunar Lander. However, the trade-off for this stability is a slower convergence rate, as the algorithm takes smaller steps in updating the policy, which is evident from the longer training times observed in our experiments.

Furthermore, the scalability of A3C, while beneficial in reducing overall training time through parallelism, also introduces challenges in terms of synchronizing the updates across multiple workers. In scenarios where the environment

is highly dynamic or requires precise timing, this lack of synchronization could lead to suboptimal policy updates. On the other hand, PPO's iterative and synchronized approach, though slower, is more robust to such complexities, making it better suited for environments with intricate dynamics.

VI. CONCLUSION

In conclusion, we have conducted an analysis of the performance of A3C and PPO algorithms in two simulated environments: Cart Pole and Lunar Lander. The results obtained indicate that A3C generally achieves faster training times, although with greater instability in the reward values. On the other hand, PPO exhibits a more stable training process, albeit with longer execution times. These findings suggest that the choice of algorithm should be based on the specific requirements of the application, considering the trade-off between training time and stability.

The results of this study suggest that A3C is a suitable choice for applications where fast training times are critical, even if this comes at the expense of some stability. PPO, on the other hand, is a better option for applications that require a more stable training process, even if this means longer execution times.

Although our study offered insights into how A3C and PPO algorithms perform in CartPole and Lunar Lander environments, future research could expand these analyses to encompass a broader range of environments. Exploring environments with different characteristics, such as those with higher-dimensional state spaces or more complex dynamics, could offer a more comprehensive understanding of these algorithms' capabilities. In addition, another promising direction could involve deploying these algorithms in practical applications in real-world scenarios, such as robotic control, autonomous driving, or financial modeling, where the trade-offs between training speed and stability are critical. In particular, the scalability of A3C makes it well-suited for applications that require rapid learning in distributed systems, while the stability of PPO could be advantageous in safety-critical domains where consistent performance is essential.

Investigating modifications or hybrid approaches that combine the strengths of both A3C and PPO could lead to the development of more efficient learning algorithms. Future research could explore enhancements like alternate exploration strategies, incorporation of new memory mechanisms, or novel regularization techniques. Finally, the integration of transfer learning techniques could be explored to enhance the applicability of these algorithms in scenarios where pre-trained models need to be adapted to new tasks or environments.

Our study focused on A3C and PPO, but other RL algorithms, such as DQN, TRPO, or Soft Actor-Critic (SAC), also hold significant importance in research. Future work could involve conducting similar comparative analyses with these algorithms, which may offer different advantages in terms of stability, convergence speed, or learning efficiency.

REFERENCES

- [1] G. Lample and D. S. Chaplot, "Playing fps games with deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, 2017, pp. 1–7.
- [2] Y. Li, X. Wang, H. Liu, L. Pu, S. Tang, G. Wang, and X. Liu, "Reinforcement learning-based resource partitioning for improving responsiveness in cloud gaming," *IEEE Trans. Comput.*, vol. 71, no. 5, pp. 1049–1062, May 2022.
- [3] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.
- [4] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, Nov. 2019.
- [5] S. Miyashita, X. Lian, X. Zeng, T. Matsubara, and K. Uehara, "Developing game AI agent behaving like human by mixing reinforcement learning and supervised learning," in *Proc. 18th IEEE/ACIS Int. Conf. Softw. Eng., Artif. Intell., Netw. Parallel/Distrib. Comput. (SNPD)*, Jun. 2017, pp. 489–494.
- [6] A. Charpentier, R. Élie, and C. Remlinger, "Reinforcement learning in economics and finance," *Comput. Econ.*, vol. 62, no. 1, pp. 425–462, Jun. 2023.
- [7] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Math. Finance*, vol. 33, no. 3, pp. 437–503, Jul. 2023.
- [8] Y. Sasaki, S. Matsuo, A. Kanezaki, and H. Takemura, "A3C based motion learning for an autonomous mobile robot in crowds," in *Proc. IEEE Int. Conf. Syst., Man Cybern. (SMC)*, Oct. 2019, pp. 1036–1042.
- [9] A. A. Shahid, L. Roveda, D. Piga, and F. Braghin, "Learning continuous control actions for robotic grasping with reinforcement learning," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2020, pp. 4066–4072.
- [10] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 31–36.
- [11] A. D. del Río, J. Serrano, D. Jimenez, L. M. Contreras, and F. Alvarez, "A deep reinforcement learning quality optimization framework for multimedia streaming over 5G networks," *Appl. Sci.*, vol. 12, no. 20, p. 10343, Oct. 2022.
- [12] A. del Río, J. Serrano, D. Jimenez, L. M. Contreras, and F. Alvarez, "Multisite gaming streaming optimization over virtualized 5G environment using deep reinforcement learning techniques," *Comput. Netw.*, vol. 244, May 2024, Art. no. 110334.
- [13] G. Dhiman, A. V. Kumar, R. Nirmalan, S. Sujitha, K. Srihari, N. Yuvaraj, P. Arulprakash, and R. A. Raja, "Multi-modal active learning with deep reinforcement learning for target feature extraction in multi-media image processing applications," *Multimedia Tools Appl.*, vol. 82, no. 4, pp. 5343–5367, Feb. 2023.
- [14] A. Al-Jawad, I.-S. Comsa, P. Shah, O. Gemikonakli, and R. Trestian, "An innovative reinforcement learning-based framework for quality of service provisioning over multimedia-based SDN environments," *IEEE Trans. Broadcast.*, vol. 67, no. 4, pp. 851–867, Dec. 2021.
- [15] E. Mocanu, D. C. Mocanu, P. H. Nguyen, A. Liotta, M. E. Webber, M. Gibescu, and J. G. Slootweg, "On-line building energy optimization using deep reinforcement learning," *IEEE Trans. Smart Grid*, vol. 10, no. 4, pp. 3698–3708, Jul. 2019.
- [16] L. Yu, S. Qin, M. Zhang, C. Shen, T. Jiang, and X. Guan, "A review of deep reinforcement learning for smart building energy management," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 12046–12063, Aug. 2021.
- [17] S. Yun, J. Choi, Y. Yoo, K. Yun, and J. Y. Choi, "Action-decision networks for visual tracking with deep reinforcement learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1349–1358.
- [18] P. D. A. Afonso and P. R. Ferreira, "Autonomous navigation of wheelchairs in indoor environments using deep reinforcement learning and computer vision," in *Proc. Latin Amer. Robot. Symp. (LARS), Brazilian Symp. Robot. (SBR), Workshop Robot. Educ. (WRE)*, Oct. 2023, pp. 260–265.
- [19] G. Zheng, F. Zhang, Z. Zheng, Y. Xiang, N. J. Yuan, X. Xie, and Z. Li, "DRN: A deep reinforcement learning framework for news recommendation," in *Proc. World Wide Web Conf.*, 2018, pp. 167–176.
- [20] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement learning to optimize long-term user engagement in recommender systems," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 2810–2818.
- [21] L. Huang, M. Fu, F. Li, H. Qu, Y. Liu, and W. Chen, "A deep reinforcement learning based long-term recommender system," *Knowl.-Based Syst.*, vol. 213, Feb. 2021, Art. no. 106706.
- [22] S. Milz, G. Arbeiter, C. Witt, B. Abdallah, and S. Yogamani, "Visual SLAM for automated driving: Exploring the applications of deep learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2018, pp. 247–257.
- [23] B. Osinski, A. Jakubowski, P. Ziecina, P. Milos, C. Galias, S. Homoceanu, and H. Michalewski, "Simulation-based reinforcement learning for real-world autonomous driving," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 6411–6418.
- [24] J. Chen, B. Yuan, and M. Tomizuka, "Model-free deep reinforcement learning for urban autonomous driving," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Oct. 2019, pp. 2765–2771.
- [25] H. Chen, K.-L. Chen, H.-Y. Hsu, and J.-Y. Hsieh, "An adaptive federated reinforcement learning framework with proximal policy optimization for autonomous driving," in *Proc. IEEE 5th Eurasia Conf. IoT, Commun. Eng. (ECICE)*, Oct. 2023, pp. 331–335.
- [26] D. Xu, P. Qiao, and Y. Dou, "Aggregation transfer learning for multi-agent reinforcement learning," in *Proc. 2nd Int. Conf. Big Data Artif. Intell. Softw. Eng. (ICBASE)*, Sep. 2021, pp. 547–551.
- [27] C. Yu, J. Liu, S. Nemati, and G. Yin, "Reinforcement learning in healthcare: A survey," *ACM Comput. Surv.*, vol. 55, no. 1, pp. 1–36, Jan. 2023.
- [28] Y. Dai, G. Wang, K. Muhammad, and S. Liu, "A closed-loop healthcare processing approach based on deep reinforcement learning," *Multimedia Tools Appl.*, vol. 81, no. 3, pp. 3107–3129, Jan. 2022.
- [29] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, and J. Schulman, "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.
- [30] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, "Fine-tuning language models from human preferences," 2020, *arXiv:1909.08593*.
- [31] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, M. F. Balcan and K. Q. Weinberger, Eds., New York, NY, USA, Jun. 2016, pp. 1928–1937.
- [32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [33] D. Reda, T. Tao, and M. van de Panne, "Learning to locomote: Understanding how environment design matters for deep reinforcement learning," in *Proc. 13th ACM SIGGRAPH Conf. Motion, Interact. Games*, New York, NY, USA, 2020, pp. 1–10.
- [34] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai GYM," 2016, *arXiv:1606.01540*.
- [35] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-13, no. 5, pp. 834–846, Sep. 1983.
- [36] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [37] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th Conf. Artif. Intell. (AAAI)*, Mar. 2016, pp. 2094–2100.
- [38] Y. Zhang, P. Sun, Y. Yin, L. Lin, and X. Wang, "Human-like autonomous vehicle speed control by deep reinforcement learning with double Q-learning," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2018, pp. 1251–1256.
- [39] Y. Yuan, Z. L. Yu, Z. Gu, Y. Yeboah, W. Wei, X. Deng, J. Li, and Y. Li, "A novel multi-step Q-learning method to improve data efficiency for deep reinforcement learning," *Knowl.-Based Syst.*, vol. 175, pp. 107–117, Jul. 2019.
- [40] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," 2017, *arXiv:1502.05477*.
- [41] Y. Zhang and K. W. Ross, "On-policy deep reinforcement learning for the average-reward criterion," in *Proc. 38th Int. Conf. Mach. Learn.*, vol. 139, M. Meila and T. Zhang, Eds., Jul. 2021, pp. 12535–12545.
- [42] W. Meng, Q. Zheng, Y. Shi, and G. Pan, "An off-policy trust region policy optimization method with monotonic improvement guarantee for deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 5, pp. 2223–2235, May 2022.

- [43] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, Jun. 2013.
- [44] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 5026–5033.
- [45] D. Ernst, M. Glavic, F. Capitanescu, and L. Wehenkel, "Reinforcement learning versus model predictive control: A comparison on a power system problem," *IEEE Trans. Syst., Man, Cybern., B*, vol. 39, no. 2, pp. 517–529, Apr. 2009.
- [46] Y. Lin, J. McPhee, and N. L. Azad, "Comparison of deep reinforcement learning and model predictive control for adaptive cruise control," *IEEE Trans. Intell. Vehicles*, vol. 6, no. 2, pp. 221–231, Jun. 2021.
- [47] A. Jeerige, D. Bein, and A. Verma, "Comparison of deep reinforcement learning approaches for intelligent game playing," in *Proc. IEEE 9th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2019, pp. 0366–0371.
- [48] S. Nagendra, N. Podila, R. Ugarakhod, and K. George, "Comparison of reinforcement learning algorithms applied to the cart-pole problem," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Sep. 2017, pp. 26–32.
- [49] H. Arabnejad, C. Pahl, P. Jamshidi, and G. Estrada, "A comparison of reinforcement learning techniques for fuzzy cloud auto-scaling," in *Proc. 17th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGRID)*, May 2017, pp. 64–73.
- [50] S. Padakandla, "A survey of reinforcement learning algorithms for dynamically varying environments," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–25, Jul. 2021.
- [51] Y. Liu, K. L. Man, G. Li, T. R. Payne, and Y. Yue, "Evaluating and selecting deep reinforcement learning models for OptimalDynamic pricing: A systematic comparison of PPO, DDPG, and SAC," in *Proc. 8th Int. Conf. Control Eng. Artif. Intell.*, New York, NY, USA, Jan. 2024, pp. 215–219.
- [52] M. Vishal, Y. Satija, and B. S. Babu, "Trading agent for the Indian stock market scenario using actor-critic based reinforcement learning," in *Proc. IEEE Int. Conf. Comput. Syst. Inf. Technol. for Sustain. Solutions (CSITSS)*, Dec. 2021, pp. 1–5.
- [53] M. Patil, B. Wehbe, and M. Valdenegro-Toro, "Deep reinforcement learning for continuous docking control of autonomous underwater vehicles: A benchmarking study," in *Proc. OCEANS*, Sep. 2021, pp. 1–7.
- [54] F. Bahrpeyma, A. Sunilkumar, and D. Reichelt, "Application of reinforcement learning to UR10 positioning for prioritized multi-step inspection in NVIDIA omniverse," in *Proc. IEEE Symp. Ind. Electron. Appl. (ISIEA)*, Jul. 2023, pp. 1–6.
- [55] R. Ren, J. Wang, J. Yu, X. Zhu, X. Luo, and H. Lu, "Deep reinforcement learning for connection density maximization in NOMA-based NB-IoT networks," in *Proc. 8th Int. Conf. Comput. Commun. Syst. (ICCCS)*, Apr. 2023, pp. 357–361.
- [56] A. Laud and G. DeJong, "The influence of reward on the speed of reinforcement learning: An analysis of shaping," in *Proc. 20th Int. Conf. Mach. Learn.*, 2003, pp. 440–447.
- [57] K. S. Narendra, Y. Wang, and S. Mukhopadhyay, "Fast reinforcement learning using multiple models," in *Proc. IEEE 55th Conf. Decis. Control (CDC)*, Dec. 2016, pp. 7183–7188.
- [58] Y. Yu, "Towards sample efficient reinforcement learning," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 5739–5743.
- [59] D. Yarats, A. Zhang, I. Kostrikov, B. Amos, J. Pineau, and R. Fergus, "Improving sample efficiency in model-free reinforcement learning from images," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 10674–10681.
- [60] J. Zhang, J. Kim, B. O'Donoghue, and S. Boyd, "Sample efficient reinforcement learning with reinforce," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 10887–10895.
- [61] M. Han, L. Zhang, J. Wang, and W. Pan, "Actor-critic reinforcement learning for control with stability guarantee," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6217–6224, Oct. 2020.
- [62] S. A. Khader, H. Yin, P. Falco, and D. Kragic, "Stability-guaranteed reinforcement learning for contact-rich manipulation," *IEEE Robot. Autom. Lett.*, vol. 6, no. 1, pp. 1–8, Jan. 2021.
- [63] L. Buşoniu, T. de Bruin, D. Tolić, J. Kober, and I. Palunko, "Reinforcement learning for control: Performance, stability, and deep approximators," *Annu. Rev. Control*, vol. 46, pp. 8–28, Jan. 2018.
- [64] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," 2016, *arXiv:1611.02167*.
- [65] Y. Jaafra, J. L. Laurent, A. Deruyver, and M. S. Naceur, "Reinforcement learning for neural architecture search: A review," *Image Vis. Comput.*, vol. 89, pp. 57–66, Sep. 2019.
- [66] P. Ladosz, L. Weng, M. Kim, and H. Oh, "Exploration in deep reinforcement learning: A survey," *Inf. Fusion*, vol. 85, pp. 1–22, Sep. 2022.
- [67] A. Y. Majid, S. Saaybi, V. Francois-Lavet, R. V. Prasad, and C. Verhoeven, "Deep reinforcement learning versus evolution strategies: A comparative survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11939–11957, Sep. 2024.
- [68] P. Vamplew, R. Dazeley, and C. Foale, "Softmax exploration strategies for multiobjective reinforcement learning," *Neurocomputing*, vol. 263, pp. 74–86, Nov. 2017.
- [69] C. Hu and M. Xu, "Adaptive exploration strategy with multi-attribute decision-making for reinforcement learning," *IEEE Access*, vol. 8, pp. 32353–32364, 2020.
- [70] Y. Achbany, F. Fouss, L. Yen, A. Pirotte, and M. Saerens, "Tuning continual exploration in reinforcement learning: An optimality property of the Boltzmann strategy," *Neurocomputing*, vol. 71, nos. 13–15, pp. 2507–2520, Aug. 2008.
- [71] Z.-X. Xu, X.-L. Chen, L. Cao, and C.-X. Li, "A study of count-based exploration and bonus for reinforcement learning," in *Proc. IEEE 2nd Int. Conf. Cloud Comput. Big Data Anal. (ICCCBDA)*, Apr. 2017, pp. 425–429.
- [72] N. Tasfi and M. Capretz, "Noisy importance sampling actor-critic: An off-policy actor-critic with experience replay," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–8.
- [73] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, and N. de Freitas, "Sample efficient actor-critic with experience replay," 2017, *arXiv:1611.01224*.
- [74] Z. Wang and H. Wang, "GAN-based PPO-LSTM high-speed railway line optimization," in *Proc. 3rd Int. Conf. Digit. Soc. Intell. Syst. (DSInS)*, Nov. 2023, pp. 125–130.
- [75] Y. Zhang, T. Gao, and Q. Mi, "A robust offline reinforcement learning algorithm based on behavior regularization methods," in *Proc. IEEE Int. Conf. Ind. 4.0, Artif. Intell., Commun. Technol. (IAICT)*, Jul. 2022, pp. 150–154.
- [76] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13344–13362, Nov. 2023.
- [77] H. Ju, R. Juan, R. Gomez, K. Nakamura, and G. Li, "Transferring policy of deep reinforcement learning from simulation to reality for robotics," *Nature Mach. Intell.*, vol. 4, no. 12, pp. 1077–1087, Dec. 2022.
- [78] T. Yang, J. Hao, Z. Meng, Z. Zhang, Y. Hu, Y. Cheng, C. Fan, W. Wang, W. Liu, Z. Wang, and J. Peng, "Efficient deep reinforcement learning via adaptive policy transfer," 2020, *arXiv:2002.08037*.
- [79] M. E. Taylor and P. Stone, "Behavior transfer for value-function-based reinforcement learning," in *Proc. 4th Int. Joint Conf. Auto. Agents Multiagent Syst.*, Jul. 2005, pp. 53–59.
- [80] M. E. Taylor, N. K. Jong, and P. Stone, "Transferring instances for model-based reinforcement learning," in *Machine Learning and Knowledge Discovery in Databases (Lecture Notes in Computer Science)*, vol. 5212, W. Daelemans, B. Goethals, and K. Morik, Eds. Berlin, Germany: Springer, 2008, doi: [10.1007/978-3-540-87481-2_32](https://doi.org/10.1007/978-3-540-87481-2_32).
- [81] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2020, pp. 737–744.
- [82] Y. Sun, K. Zhang, and C. Sun, "Model-based transfer reinforcement learning based on graphical model representations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 2, pp. 1035–1048, Feb. 2023.
- [83] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester, "Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis," *Mach. Learn.*, vol. 110, no. 9, pp. 2419–2468, Sep. 2021.
- [84] M. Cutler, T. J. Walsh, and J. P. How, "Real-world reinforcement learning via multifidelity simulators," *IEEE Trans. Robot.*, vol. 31, no. 3, pp. 655–671, Jun. 2015.
- [85] M. Genkin and J. J. McArthur, "Using reinforcement learning with transfer learning to overcome smart building cold start," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, Dec. 2022, pp. 713–718.
- [86] L. Ji, Q. Qin, B. Han, and H. Yang, "Reinforcement learning to optimize lifetime value in cold-start recommendation," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2021, pp. 782–791.

- [87] Q. Hou and J. Dong, "Cooperative output regulation of linear multiagent systems with parameter convergence," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 1, pp. 518–528, Jan. 2023.
- [88] Q. Hou and J. Dong, "Improved event-triggered prescribed-time cooperative output regulation for nonlinear multiagent systems," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 4, pp. 2274–2278, Apr. 2024.
- [89] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, no. 1, pp. 237–285, Jan. 1996.



ALBERTO DEL RIO received the degree in mobile and space communications engineering from the Universidad Carlos III of Madrid (UC3M), and the master's degree in signal processing and machine learning for big data from the Universidad Politécnica de Madrid (UPM), where he is currently pursuing the Ph.D. degree. He was with Deutsche Telekom, Berlin, in the specification of the standard of 5G telecommunications system and release 16. Specifically, in the development of a system framework concept focused on cloud services. He is with the Grupo de Aplicación de Telecomunicaciones Visuales (GATV), UPM, in dedicated projects on 5G communications networks with the help of use cases focused on AI.



DAVID JIMENEZ received the Telecom Engineer degree and the Telecom Ph.D. degree (cum laude) from the Universidad Politécnica de Madrid (UPM), in 2004 and 2012, respectively. In 2004, he joined GATV, as a Researcher. He is an Assistant Professor with the Department of Physical Electronics, Electrical Engineering and Applied Physics. He is the author or co-author of several scientific publications. His main research activities are focused on 5G communications, media quality and quality of experience (QoE) assessment, smart energy, and electric vehicles. He is part of the Executive Board of the Chair of RTVE, UPM. He is a reviewer of several entities both for research and development certification and scientific publications.



JAVIER SERRANO received the degree in sound and image engineering from the Universidad Politécnica de Madrid (UPM), the master's degree (by research) in signal, image, speech and telecommunications from the Institut Polytechnique (INP) de Grenoble, France, and the Telecom Ph.D. degree (cum laude) from UPM, in 2023. He is currently with the Grupo de Aplicación de Telecomunicaciones Visuales (GATV), UPM, on testing and validation of new UHD and immersive content delivery models in next-generation mobile networks. He is also an Assistant Professor with the Informatic Systems Technical School of UPM, specialized on advanced networks and networks management.

• • •